Twenty twenty vision — software architectures for intelligence into the 21st century

P A Martin

This paper addresses the need for enterprise-wide software systems that are easy to maintain and tolerant of hardware failure. This need is discussed in the context of a telephony service enterprise. An approach is proposed that uses distribution and object oriented techniques. Current enabling technology is considered and it is argued that a different approach would produce more elegant systems.

1. Introduction

The communications networks of the future will support high-bandwidth, low-delay information flow. The likely application areas to take advantage of these gigabit per second networks are video-on-demand, animated shared simulations and globally distributed computing. As corporations grow to attain global reach, the scale of their computing and communications infrastructures will also grow. This paper explores potential future software architectures that would support these new widely distributed applications. It then compares the needs of such architectures with the promise held in emerging distributed computing technologies.

The software architectures proposed are based on the assumption of computer level integration of the communications industry and their customers. For the purposes of this paper, customers as viewed by the communications industry come in two types. The first is the end-user and is the consumer of services. The second provides a communications-based service to its customers, who are the previously described end-users. This group is referred to as the service provider. This paper assumes that service providers will have access to facilities that have previously been beyond their control. For instance, they will be able to establish connections between interested parties and to control the flow of information between them.

The paper then goes on to explore what these groups have in common and seeks to model them as enterprises. The use of large shared-object systems to model an IN service provision enterprise is explored in detail. The examples given are set in traditional telecommunications intelligent network applications. The use of such object systems is as applicable to communications enterprises as it is to service providers or any other enterprise. Large shared-object systems are expected to form the fundamental building block of computer infrastructures of the future.

2. Assumed physical architecture

It is not the purpose of this paper to suggest divisions of responsibility between different enterprises contributing to the provisioning of future IN services. Rather the focus is on flexible software architectures that can be used in any division of responsibilities. Accordingly the following description of an assumed architecture is more by way of example. Figure 1 shows a likely overall physical architecture and each domain is described below.

2.1 The role of the transport domain

In the centre of the diagram is the transport domain which includes the physical network infrastructure such as switches and fibre optic links.

This domain performs the task of delivering information from one end-point to another end-point. The delivery can either be connection oriented or connectionless. With connection-oriented delivery streams of information flow from one end-point to the other — so this is typically suited to playing a film once the equipment at either end is set up

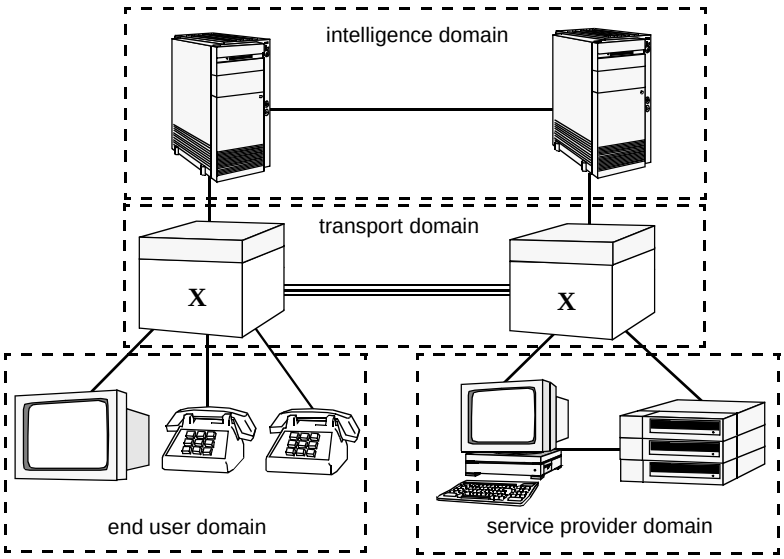

Fig 1 Assumed physical architecture.

to either play or receive the flow. Connectionless delivery is used more in the messaging that is used to set up the connection. Accordingly the transport mechanism must be able to support both delivery methods.

For the purposes of the paper, this domain includes the processing that controls which end-point is connected to which other end-point and with which bandwidth. This connection control activity is similar to call control in existing networks. However, the call concept is at a higher level and more telecommunications specific. Software that uses connection control, such as the intelligence software, may specify end-points to connect between, and be informed of, the progress of the connection.

2.2 The role of the service provider domain

The service providers are enterprises who advertise their services. They then have equipment ready awaiting connection requests. End users then dial in and connect to the service providers' equipment. A typical role would be providing video services. Accordingly they are shown in Fig 1 with a computer and a video player. Having connected to a specific service provider, the end user would then interact with an application running on the service provider's computer in order to select a film. The appropriate bearer would then be set up across the transport domain and the film played to the end user's TV.

Traditional IN applications or services such as number translation and time of day routeing revolve around basic telephony linked to personalised customer data. Where do such applications fit into the software architectures of the future? Are they applications to be provided by the service providers at the network edge or are they so closely linked to the switching that they have to be near the core in the intelligence? This is discussed in Swale and Chesterman [1].

2.3 The role of the end users

End users are similar to service providers in so much as they are both customers of the transport provider. They connect their equipment into the transport infrastructure and then use it to dial up services as provided by the service providers. Their role is therefore to consume services and transport bandwidth. Figure 1 shows end users equipped with appropriate apparatus such as television sets.

2.4 The role of the intelligence domain

Having described the role of the other three components of Fig 1, the question arises: 'What remains for the intelligence domain to do?'

The times at which the intelligence domain becomes active seem to revolve around call set-up and clearing which may provide a rationale for deciding where specific services are best located. The traditional IN services, such as number translation and time-of-day routeing, which come into play before a call is established fit naturally into the intelligence domain. In addition to these well-understood applications other opportunities exist for the intelligence domain to make use of its position between end user and service provider.

The intelligence domain is aware of the other domains and can hold the necessary information to connect them together. The intelligence domain may provide advertising facilities. Service providers would advertise their services and end users may view and select from them. This is

similar to the current 'Yellow Pages' service. Having located the appropriate service provider the intelligence domain would route further signalling through to them. The selection of services is likely to be automated to some degree in the future, making use of personalised software agents. An end user would load its agent with a query and it could inquire of various service providers about the options available. The results would then be delivered back to the end user for them to select an option.

Also, given numerous service providers wishing to bill their customers, an opportunity exists for intelligence to offer integrated billing services.

3. A software model for intelligence services

The software architectures that are best suited to controlling these systems are now assessed. The foregoing text described a number of groups which use the transport domain. These are end users, service providers and intelligence. What these groups have in common is that they can be viewed as separate enterprises with their own supplier-customer relationships. This paper seeks to recognise what enterprises in general require of a computing infrastructure with special emphasis on the needs of IN. In order to understand the requirements that enterprises expect their suppliers to fulfil it is useful to develop a model of those enterprises.

Enterprises can consist of single individuals going about their business or, at the other end of the scale, they may be multinational corporations. From the communications company point of view, the enterprises that use their facilities must be understood so that any barriers to their expanded use can be removed. Communications companies are themselves enterprises and can also benefit from such an analysis.

3.1 Why model the enterprise?

Everyone who is involved in an enterprise carries a model of that enterprise around with them in their head. By example, if the enterprise is a window cleaner's round then the only people involved are the cleaner and the clients. The cleaner can remember the clients, their requirements and how much they owe. If anyone wants to know anything about the enterprise they must ask the cleaner. Sadly, few enterprises nowadays are this simple. A successful window cleaner would have taken on employees, more cleaners, accountants and the like. The model becomes more complex. External agencies, such as government departments, have to be comprehended and integrated into the model. If the enterprise becomes large enough, it is beyond the capacity of any one individual and this is where the trouble starts.

Typically, each part of the enterprise will have been divided up into separate traditional departments. The accounts department handles the finances and keeps their records based on the fiscal components of the enterprise. The roundsmen keep their records based on the requirements of their round and so forth. These records have a number of components in common but are based on the functions performed by the separate groups. They are maintained separately and hence evolve into working systems with defined, probably paper-based, interfaces to other departments.

Automation of the record-keeping process usually serves to further define the functional, departmental nature of the enterprise. Islands of automation arise and interface systems are produced to provide integration. The result is that each enterprise maintains multiple computer systems each with their own overlapping models of parts of the enterprise.

It has therefore become evident that another approach could be more profitable. If it is possible to model the enterprise as a whole then considerable benefits can be attained. For instance any user can explore the running and data of the enterprise without the restriction of department barriers.

The situation is no different in the world of IN and service provision. Successful services have been installed as essentially stand-alone systems, needing interfaces to other systems. Telecommunications companies have been striving to introduce generalised systems and the remainder of this paper suggests how this could be approached by modelling the provision of IN services in objects.

3.2 Why use objects for IN?

A number of technologies are available to enterprises that wish to model their activities. The dominant software methodology for enterprise modelling is likely to be object orientation (OO) [2—4]. Objects as a software concept are now entering the mainstream for new application development because they offer significant promise to overcome the barriers that hold enterprises back. Increasingly enterprises are restricted by their inability to quickly and safely evolve their software systems to meet new needs and opportunities. So what benefits do objects hold for enterprises in general and IN service providers specifically?

The most important benefit of an OO system is the encapsulation of software components. This is a form of automatic modularisation that means the software component publishes a fixed interface. Behind the interface the actual implementation can be changed. This has the effect of dividing up the software into handleable-size chunks so that defects in one chunk do not pollute other software com-

ponents. As, in reality, the definition of the published interface is rarely unambiguous, misinterpretations of an object's duties often lead to inelegant systems. This is more an issue of software engineer discipline than a failing of OO methodologies.

Another significant benefit of using objects is that they lend themselves to the modelling of real-life systems, especially systems that support the 'is-a-type-of' relationship via inheritance. Theoretically the one-to-one relationship between the real world and its model in software objects would lead to easy reuse of software components. There is currently little evidence of extensive and successful software reuse but this is again an issue of software engineer discipline.

These benefits apply to IN applications as they do to any other application domain. Objects are therefore a powerful methodology to adopt for future systems but strict control needs to be used in their deployment if their benefits are to be experienced.

3.3 Why use distributed shared objects for IN?

Having accepted the benefits of modelling the enterprise and using objects, then why add the extra complication of using distributed objects? The most compelling reasons in IN applications are model size and availability. Once an enterprise model gets to a certain size in terms of the memory, disk and CPU usage requirements it can no longer be hosted on only one computer. For example, an enterprise that provides IN services may have many millions of customers, each requiring their own personalised data to be ready for use whenever they invoke a service.

Additionally, in a single host system, if that one computer is unavailable, then the IN service as a whole is also unavailable and hence losing money. Distributed objects bring the promise of truly huge, continuously and globally available applications. Unfortunately this is only a promise at the moment as genuinely useful large object systems are not currently available.

An obvious but awkward requirement of large object systems is that many applications will be simultaneously accessing and editing the objects. For instance a billing application may want to read customer object data whilst a customer-handling application is updating it. Conflicts are bound to arise and the system should resolve these in an equitable manner without allowing the model to get into an inconsistent state. This means that the model has to include some form of object locking or transactional system.

In addition to the general requirements for large shared object systems, their use in IN applications apply extra criteria. These include high availability, near real time response and very wide area distribution. These themes are picked up in section 4.

3.4 An example object model for intelligent metworks

For a typical model of IN services the transport domain would support objects such as switches and ports. The objects supported by the intelligence domain would include devices, such as telephones and televisions, customers, calls and services. A simplified version of the model is given in Fig 2, which shows the model that could be used to control the transport and intelligence domains of the assumed physical architecture shown in Fig 1. It should be noted that there is a one-to-one mapping between a real world entity and an object in the model.

The transport domain would also support the switch and port objects. Each physical switch that the provider owns would be represented by a switch object in their object model. This means that the transport domain would need its own computing infrastructure. The switch and port objects could publish their interfaces to the other domains. Accordingly the intelligence domain could invoke operations on the switch object resulting in actions on the real switch and thus actions on the real world devices connected to the switches' ports. If the intelligence domain has some way of registering an interest in activity on ports then the transport domain can invoke operations on intelligence domain objects. For instance, an off-hook event may be reported back to the intelligence domain's device object using this mechanism.

The device, customer, call and service objects would reside in the intelligence domain. The data associated with a customer's time-of-day routeing (TODR) is shown embedded in the customer object as it is uniquely associated with that specific customer. These objects could export their interfaces to the other domains. Service providers could use the call-object interface to request a path suitable for playing video across the transport network. Hence the published object interfaces dictate the functionality that enterprises may request or supply to one another.

In this scenario, each domain or enterprise would have their own computing infrastructure connected to the transport infrastructure. On those computers they maintain their own enterprise models. At the edge of those models they can connect to other enterprises' models.

4. Requirements of a large shared-object model system to support IN

The requirements for supporting large shared-object models are common to many application domains. This section points out the general requirements with emphasis on the needs of IN services.

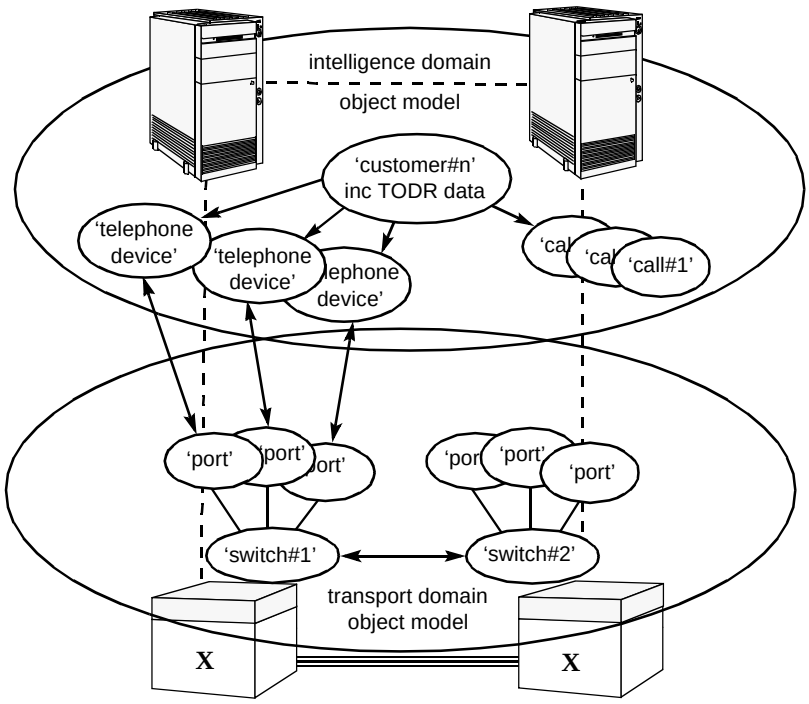

Fig 2 An example object model for IN.

There are three groups of people within each enterprise who would make demands on any system that supports object models. Any system that is to be widely adopted must satisfy their needs in a manner that is not too costly to the providers. These groups are the application users, application programmers and administrators. Their requirements are described below.

4.1 Requirements from the application user's point of view

Desire to create/destroy/read/edit objects

The application user will wish to interact with the object model as provided to them by the programmers. This would mean having the ability to create and destroy objects and to manipulate and browse those objects whilst they exist. To illustrate by example, application users could create customer objects, manipulate the attributes of that object and invoke its operations. When the customer takes his business elsewhere, the application user would delete the customer object from the system.

This activity is traditionally embedded in applications. If large-scale shared-object models are adopted, specific applications can be replaced with generalised graphical user interface (GUI) based object editors or browsers. Attribute types such as integers, floats and strings can easily be represented in screen-based forms. These fundamental types will need to be augmented with new data types that are required to accurately model enterprises. These types could be accessed from the browsers and will include pictures, video clips and sound bites.

Continuous availability

The application user will want access to their applications at any time of the day, any day of the week and any week of the year. Traditional computer environments that provide continuous availability have relied on customised hardware that provides low performance at a high price. This price is handed onto the customers that demand the availability.

From the application user's point of view availability is calculated in terms of either unplanned time without access or absolute (planned plus unplanned) time without access. Obviously both types of downtime are undesired but planned downtime is not as disruptive as unplanned downtime.

Ubiquitous availability

The application users will want to access their service from wherever they are. An enterprise that is multinational, even global, will expect global access from all its sites to all its sites. Moreover the appliances that are used to interact with the enterprise model via applications will vary from site to site. Devices for access to applications will range from simple textual or voice to multimedia personalised devices. Increasingly these appliances will be evolutions of mobile communications and computing devices [4, 5].

Adequate performance

Application performance under 'hard' real time conditions has mandated criteria. Such conditions would typically be nuclear power station or airliner control where failure to meet time limits would result in disasters. For most applications, such as IN services, the result of slow performance would be that a person or a system has to wait and the only consequence is that some time is lost. This does not mean to say that high performance is not desirable but any definition of concrete performance requirements must be treated with scepticism.

Security

The object model will contain objects and parts of objects that are not for general consumption. Enterprises will want to divulge parts of their object model to foreign users based on who that user is. For example customers should not be able to edit the account component of their own customer object. However, they may be allowed to edit their own 'time-of-day routeing' data without service provider intervention. Accordingly the object model should be equipped with access control lists for each attribute of each object.

4.2 Requirements from the application programmer's point of view

The commonly used term application programmer is probably too narrow in the sense meant here. The tasks undertaken by this group are defining and implementing the enterprise model in order to fulfil the needs of the enterprise. The skills required are commonly associated with analysts or designers. Any system that is to be widely adopted must please, even excite, this opinion-forming group. The system will fulfil the needs of this group in terms of the things they want to be able to do and, just as importantly, the things with which they actively do not want to be bothered.

Desire to create/destroy/read/edit classes

The enterprise model is likely to be implemented in class libraries. The definition of classes, what data they hold and what operations they perform, is a highly skilled task. Each class should model some concept in which the enterprise has an interest. For example, a communications company enterprise model is likely to include classes that define device and switch objects.

The programmers will want to be able to create new classes easily, to extend the object model. They will also want to be able to add new attributes and operations to existing classes. Similarly, non-useful class attributes and operations should be easily withdrawable. Although easily expressed, this is a very difficult need to fulfil. Once an object model is active, evolution of that model is very difficult in practical terms. The support for model evolution has to be built in from the earliest stage of system definition.

Location transparency

The programmers will desire location transparency which means they do not have to undertake any specific coding to ensure that their objects reside on the most appropriate hosts. Given that the object model will be made available via some computing fabric, then the programmer does not wish to give explicit instructions on how best to take advantage of that fabric. An object system could field all requests to create new objects and locate them on the most appropriate host.

The most appropriate host could be chosen on the basis of a number of criteria which are likely to be related to load balancing. A host would be selected if it has sufficient memory and disk space, if it has access to the objects run-time code and, finally, if it is faster than any other host that meets the previous criteria.

An object can be used to represent an individual and would contain many attributes. These attributes could include medical records, tax records, etc. All these attributes apply uniquely to the same person and therefore should be modelled as a single object, but it can be seen that it may be necessary to distribute them across multiple hosts. The tax authority will demand that the tax records are held on their secure hosts but may be accessible from the larger object model. Similarly, medical records, including complex data types such as X-ray images, video clips and speech annotated notes, may be held on separate hosts.

This requirement adds another level of complexity to the software if it is to be supported transparently. Alternatively, the application programmer can accept the necessity for attribute distribution and code the class libraries accordingly. This would mean defining 'medical records' as a separate object and having a reference to it from the person object.

Having accepted that an object can live anywhere, and also individual attributes of objects can be dispersed, there are conditions under which the choice of locations should be restricted by further qualifications. Switch objects may only be allowed to reside on computers that are physically linked to the switches they are controlling. Expressing the qualifications and restrictions on the locations of objects will present an awkward problem for system developers if they are attempting to achieve location transparency for the programmer.

Failure transparency

The programmer does not wish to pollute the class library source with exception-handling code for cases where a remote computer has failed to fulfil its duties. The programmer therefore desires to have failure transparency so that the system will catch and recover from failures without programmer intervention. This is obviously easier to express than fulfil. The number of potential failure modes for a single computer running a single process is large. Programmers normally have to add exception-handling code for situations such as memory exhaustion or accept the inevitable crashes. As IN services require high availability, the crashes must either be avoided or caught and fall-back mechanisms invoked. When the scenario includes multiple processes on multiple hosts on multiple networks, the combinations of failure modes grow exponentially.

From the point of view of each object sitting within a process the situation is more simple. Each object, if it is a true object, has a message-based interface. That means that all the information and stimuli it receives is via messages. If it invokes a method in a remote object, it does this by passing it a message and then it will expect a response via a message.

If a computer crashes, the objects residing on that computer become permanently unavailable. It is difficult to recover from this situation, but not impossible. The system may be able to trap the crash event and replicate the objects elsewhere. From the programmer's point of view no extra code has been undertaken and failure transparency would have been achieved. Studies along these lines are undertaken by a number of teams [6, 7].

A second, more puzzling, failure mode exists. This is where the response message from the target object simply never arrives. What can the originating object make of this non-event? Did it receive the original message? Has the return message got lost? Perhaps retransmission is the thing to do? In any event the application programmer should not be invited to get involved.

Concurrency transparency

As previously stated, the object model of the enterprise will be shared between many concurrent users. Application users will invoke operations on the objects that are public to them. During the execution of that operation, the object may invoke an operation on another object, which in turn will invoke an operation on a third object. Therefore, from a simple action by an application user, a stack of invoked objects will be built up. If a second application user were to perform an action that results in the middle object of this stack to be accessed and the second user were allowed access prior to the first user completing their transaction, then an inconsistent situation can arise.

Locking can be performed on objects to prevent the chance of the inconsistency being introduced. When the system uses object locking, the well-known deadlock situation can arise. In the example the second user will have to wait until the middle object in the first user's stack becomes free. Deadlock will occur if the first user's stack is blocked waiting for access to an object in the second user's stack.

These issues are only described here as indications of the concurrency problems with which the application programmer does not wish to become involved. The application code should remain free of exception cases to cope with concurrency conflicts.

Code-evolution transparency

When a system is installed and working it is guaranteed to be subject to bugfixing and enhancement requests. It will be essential, if the model is to continue to reflect the enterprise, that it can change with the enterprise or lapse into irrelevance.

This is the most difficult and least addressed transparency issue of all. If code evolution is not handled transparently, then the burden is put on programmers and administrators to provide evolution mechanisms. Experience shows that code evolution mechanisms have components that are both mechanistic, leading to boredom and therefore prone to errors, and complex, leading to confusion and therefore equally prone to errors. The result being that systems gain high inertia and act as a brake on the ability of an enterprise to evolve.

On the assumption that an enterprise model has been implemented and is doing good service on a day-by-day basis, a new requirement arises whereby additional data has to be stored with each customer object. New code is written and tested in isolation. Then the new code has to be introduced into the live system and the existing live customer objects have to be migrated to the new format. Whilst easy to express it is an area of computer science that receives little attention and yet consumes huge amounts of *ad hoc* software engineering effort.

4.3 Requirements from the administrator's point of view

Administrators are the group within each enterprise who perform the tasks which keep the hardware and software infrastructure going throughout their life cycles. For hardware this includes purchasing equipment, configuring it, connecting it to existing equipment, monitoring its performance, arranging repairs and eventually withdrawing it from service. For software this includes purchasing, loading, configuring and arranging access for users. Also any software failures are reported back to support groups

and corrective action is taken. It is the desire of the administrators, or at least their employers, that the effort expended on maintenance is minimal. Real world experience is often the reverse, with this group being frantically overworked. Some facets of self-managing systems that would assist with effort minimisation are now described.

Self-healing systems

What often happens when something goes wrong with software is that a stream of cryptic messages is dumped in a log file from an embedded library and service is ceased. The log file is then conveyed to remote but knowledgeable individuals who tell the administrators how to recover the system. After this process has been repeated a number of times the administrators build up their own knowledge of diagnostic procedures. These procedures work until the next release of software changes the side effects of the embedded libraries.

The previously described failure-transparency mechanisms allow systems to carry on if a single node fails. This does not avoid the need for diagnostic procedures but reduces the urgency of recovering a particular system. Service can be continued from the back-up systems without administrator intervention.

Self-tuning systems

Given that the system provides object-location transparency, it can decide where to put the objects. Presumably it will do this in such a way as to achieve optimum performance with a knowledge of the resources available. If objects have the ability to migrate at run time, they can move closer to the point where they are most often used. In an extreme case, the object could move on to the same network, into the same computer and even into the same process.

Self-installing systems

Application software usually needs to be installed on the targets on which it is to run. However, a system that holds a repository of executable code can be imagined. Whenever a new computer is added to the network, the system discovers its capabilities in terms of CPU, memory and disk space and downloads whichever executables it considers appropriate, perhaps using something like the UNIX facility ‘ftp’. From that point onwards the computer can be a host to objects for which it has appropriate code.

Requests help when required

An object system that can perform all of the above tasks is inevitably undertaking a lot of resource monitoring. If the resources it has available are not sufficient to meet optimum performance then an automated report can be made to the administrators. As a result, the administrators may enhance the configuration of the computing fabric.

5. Three-layer stack approach to solutions

This section looks at possible solutions and considers how existing and emerging technologies may help. One of the most porudctive habits that developers can adopt is to think in terms of layers. If the approach is too rigid and many layers are artificially introduced, the resultant system becomes laden with inter-layer mapping libraries. Accordingly this paper identifies only three layers which can be named applications layer, middleware layer and network layer. Figure 3 illustrates these layers.

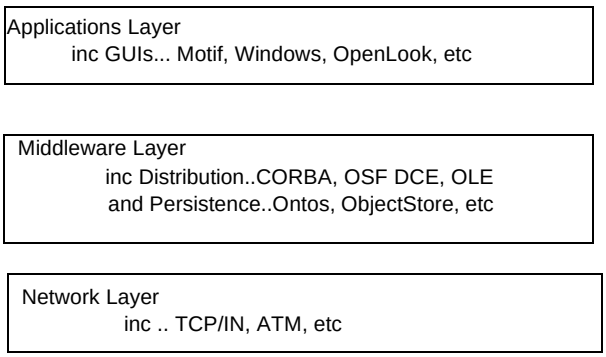

Fig 3 The three-layer stack

5.1 Existing network layer components

It is not the intention of this paper to review in detail the network technologies and products available to implementors of enterprise models, but rather to underline the shift in design criteria for higher layers of the software. Availability of high-bandwidth, low-delay networks such as ATM-based systems means that programmers need not worry as much about making savings in these areas. A failure to recognise these shifts will lead to false design decisions being made in the name of performance which will ease problems that are not bottlenecks.

5.2 Existing middleware layer components

Many products that address the requirements expressed in the previous section currently exist. The number of combinations in which these products can be connected to produce the required functionality is truly bewildering. Some of these products are considered below.

Operating systems

The operating system (OS) connects the computer software to the computer hardware and thus forms a foundation layer upon which other software is built. Previously, the relationship between hardware and

operating systems was easily defined as both were supplied by the same vendor. Increasingly, the hardware and the operating system are becoming separately supplied and are becoming standard products.

UNIX is the most popular OS for workstations and is available on a wide range of machines from supercomputers to laptops. It is also widely used in distributed applications and many significant network innovations were made in UNIX and later ported to other operating systems. There are unfortunately competing variations of UNIX but the differences are reducing.

The latest OS offering from Microsoft [8] is called Windows NT and is a true multiprocessing micro-kernel based architecture. Importantly it exports a number of operating system applications programming interfaces (APIs) which map on to a set of base facilities. Amongst these APIs are Windows and POSIX, meaning it is an OS that can run both traditional PC and UNIX applications.

Although these two are likely to be dominant in the future, other options cannot be ignored. For instance, the trailing OS/2 from IBM now seems to be making a comeback as a true contender.

Object databases

Object persistence, meaning the ability to store objects on disk and bring them into memory on an as-needed basis, is not a requirement in itself. The requirement is to support large shared-object models and object persistence is usually the mechanism by which the largeness criteria is satisfied. If another method arises that uses distributed shared memory in support of large object models, then disk-based systems may become unnecessary.

Object databases mostly operate by keeping a formatted copy of the in-memory object on disk. When an application needs to access the object the object database system will copy it into the applications address space. Each object in an object database needs its own unique name that is valid whether the object is on disk or in memory. In order to create truly large object systems a number of object databases may have to be connected together. They then have to agree on consistent object names in the wider scope. This might cause problems as object databases may be used in conjunction with object request brokers (ORBs) which have their own object-naming requirements.

Object databases are not generally considered to be sufficiently mature as yet to support mission-critical applications. Also they do not yet have the built-in querying mechanisms that have made relational systems widely acceptable.

On-line transaction processing systems

On-line transaction processing (OLTP) systems have become part of the fabric of commercial programming although their need is not always immediately recognised. A system could be written and working for weeks before a crash occurs and the data is found to be tangled. This occurs because two applications both managed to edit parts of a data structure concurrently and left the model in an inconsistent state.

The term ‘transaction’ has no direct translation into object model terminology. It is traditionally an application-level decision as to what collection of actions represents a transaction. Importantly the transaction is a collection of actions that may either be accepted as a whole or denied as a whole, leaving in either state, the data in a consistent manner. This is done by locking out other applications from the data until the transaction is complete.

If the applications of the future are customisations of generalised object browsers then it would be useful if some automatic method could be found to define application transactions. All activity in the object model is initiated by the application user invoking top level operations on public objects. Perhaps each such top-level operation is a transaction which locks out other applications until the result is delivered back to the application user.

Transaction monitor code could be built into other middleware components such as object databases but currently OLTP must exist as a separate system. Use of existing OLTP systems involves extra work for programmers at development time and a significant number of extra processes at run time.

Distribution mechanisms

Mechanisms to provide client/server programming have been receiving a lot of attention as the benefits of distributed computing have become more obvious. The term client/server refers to an architecture where the application user interacts with a computer which does some local processing but then submits a request to another computer that performs more processing with access to more centralised resources and returns the result to the requester. The application user end is called the client and the other end is the server. The client/server relationship is therefore defined by the direction of the question/answer flow.

In a distributed object model the application user will invoke an operation on an object and if that result is returned immediately then the client/server relationship can be applied. However, most often that first object will want to invoke another operation on a second object in which case the original object becomes the client and the second object the server. To add further confusion the second

object could, quite legitimately, call back to the original object to attain further information in which case the original object becomes both server and client at once. A better approach would be to say that each object was the equal or peer of the other and so no client/server relationship exists between them. Peer-to-peer architectures generally refer to those where any computer process can communicate directly with any other in order to initiate dialogues, request data and so are better suited to distributed object systems [9]. Accordingly systems that have built-in client/server assumptions will not lend themselves so readily to object modelling.

Some leading distribution mechanisms are considered below.

- OSF DCE and OODCE

  The Open Software Foundation (OSF) distributed computing environment (DCE) [10] is an integrated set of services that has been adopted by most UNIX suppliers and is primarily suitable for running in TCP/IP environments [9]. Whilst its increasingly wide, and in some cases free, availability and standardisation make it attractive to developers the functionality it provides is no more that has been available on bundled proprietary systems. However, it is another layer on top of the basic operating systems and so potentially serves to fill part of the middleware layer.

  The services that OSF DCE currently provide include a standardised remote procedure call (RPC) mechanism, security services, a useful directory service which is mostly used to locate servers, a threads library for use in servers and a distributed time service.

  OSF DCE can be considered a toolbox to help provide support for distributed object models. The toolkit is of too low a level to perform that task by itself. A Hewlett Packard (HP) product called OODCE extends the use of OSF DCE to support C++ objects and its definition is being submitted to the Object Management Group (OMG) [11] as a candidate for standardised adoption.

- CORBA

  Common Object Request Broker Architecture (CORBA) [11] is an architecture defined by the OMG which is a cross-industry consortia with some 400 members. CORBA contains a definition by which distributed object systems can be built. Current CORBA implementations provide host independence and some location transparency. The major co-developers of this standard are HP, DEC and Sun, all of which have their own offerings. The early lack of definition in the CORBA standard means that offerings from different suppliers will not interoperate, but this is expected to be fixed for future product releases. Other transparencies such as failure transparency are also expected to be introduced but not in the near term. This activity has weighty industry support and holds genuine potential as a tool to support distributed objects.

  ORBs have a better approach than object databases with respect to the instantiation of objects. With an object database each application that wants to execute an operation on an object will get a copy of that object in its own address space. This can lead to one object copy per application invocation. The concurrency problems and duplicated resource usage to which this leads must be borne by the object database management system. The ORB takes a wiser approach for large object systems. The operation invocation is sent to the object wherever it currently resides. A combination of ORB and object database systems may be an approach for widely dispersed object models.

- OLE

  Microsoft [8] support their own distributed object model called Object Linking and Embedding (OLE). Their strength in the desktop market-place has led to attempts to allow the interworking of OLE and CORBA-based applications. Microsoft and DEC are collaborating on a product called the Common Object Model (COM) that allows CORBA objects and OLE objects to co-operate.

5.3 Existing applications layer components

As previously argued, the role of applications in a shared object environment can be reduced to a generalised object editor or browser. Each application would merely be a different restriction of the view of the object model based on who was using the browser.

Graphical user interface toolkits are often tied to their supplied operating system. However, OSF Motif [10] seems to be becoming the standard GUI provided with UNIX workstations and is available on virtually every client platform. This even includes the platform from Sun which has developed a serious competitor to Motif called OpenLook [12]. Apple, Microsoft and IBM (Presentation Manager on OS/2) all provide their own GUI programmers environments.

Browsers are graphical front ends to the object model and can replace the need for writing specific applications if they are powerful enough. Object database products such as ONTOS [13] and ObjectStore [14] are supplied with built-in browsers as are many relational systems.

6. A unified approach to middleware

The discussion so far has established the desirability and requirements of large shared-object models. This was followed by a description of the current and envisaged products that could form parts of the jigsaw. The problem is that in most cases they are parts that were originally designed to fit into different jigsaws. Fitting a selection of these components together from the existing choice will undoubtedly fill the middleware gap, but at what cost?

In order to produce a complete system an architect would have to pick an interworking collection of products and then build a team with the appropriate skills. Not only would these products need to be understood in isolation but the interactions and interfaces between them would need to be understood.

This would divert effort from the real issues of defining the enterprise model into the details and constraints of various packages.

The real problem is that the packages are mostly not specifically written to support large shared-object models but are enhanced or evolved to perform this task, which means they bring excess software with them. A fresh look at the problem would bring different and probably more elegant solutions. If the system is purely to support objects then some of the necessary facilities can be built into those objects. The following paragraphs describe some ways in which the objects themselves could be empowered so they can contribute to large shared systems.

6.1 Empowerment of the objects with embedded performance tuning

Objects could for themselves keep track of which other objects are invoking their operations. In the case of a customer object which is resident on a computer in Hull and is constantly being accessed from computers in Devon each object could keep a table of its last 20 operations and decide whether to migrate to a computer closer to the invoking computer. In the extreme it could migrate on to the same computer and into the same process as the invoking object. The performance could then approach local performance. This is an example of how automatic performance tuning could be built into the middleware and hence avoid the need for human-driven tuning applications.

If the middleware supports cloned objects at a physical level, the tuning options become greater and more complex. The cloning of objects is primarily for resilience and requires that clones reside on different hosts. This acts against the interests of performance tuning and may lead to some interesting solution optimisations.

6.2 Empowerment of the objects with embedded transaction locking

Currently transaction locking is performed by external processes that monitor the progress of applications and enable ‘roll back’ or ‘commit’ as necessary. A more elegant approach would be to build an understanding of the need for transaction locking into each object. The object would ‘know’ not to commit the changes that an operation invocation may have produced until it received a message from the top-level object that the whole operation had been successfully performed. The object would also know how to roll back to the pre-invocation state if the transaction as a whole had failed.

6.3 Empowerment of the objects with the ability to display

Each object could collaborate with the browsers that are trying to display them by supporting an operation such as ‘display’ which when invoked would cause the object to make an image of itself appear on an appropriate screen. The object would need to have sufficient knowledge of its type and the security access that the invoker is allowed. Given this knowledge, general-purpose code could be written to present the object using the local GUI package.

6.4 Empowerment of the objects with failure recovery

It is difficult to see how the object that fails can arrange to recover without the application programmer, or even end user, having to intervene. If the object system employs some object replication, also known as ‘groups’, then a scheme becomes possible. With groups only one object exists from the programmer’s and end user’s points of view; however, the implementor of the middleware will map this appearance to a number of physical objects. Each object should be an exact copy of the other in which case care must be taken that any edits to one of the groups are reflected to the others. In the event of a failure one member of the group becomes unavailable. The survivors of the group, who will be physically remote from the failure, could detect the absence of their erstwhile member. Recovery would be undertaken which would include generating another replica object on another trusted computer.

7. Conclusions

This paper has indicated that large shared-object systems are likely to be the dominant software technology for IN applications and for enterprises in general. For use in an IN arena the object system needs to be highly available, widely distributed and have near real time performance.

As computing and network infrastructure increases in performance whilst dropping in price the software design criteria emphasis must shift away from speed to consistent modelling and evolvability. What will hold back enterprises of the future is their inability to evolve their software safely and quickly. A successful large shared-object system will also support some form of code evolution.

Building the power to achieve the required functionality into the objects themselves appears to be the most promising approach but may lead to some conflicts. For instance, the methods proposed to achieve concurrency transparency may conflict with the mechanism for location transparency. For this reason a holistic approach needs to be adopted which hopefully will result in a unified elegant solution.

Acknowledgements

The author would like to thank the many contacts at BT Laboratories for their comments and insights that have contributed to the paper.

References

1 Swale R P and Chesterman D R: 'Distributed intelligence and data in public and private networks', BT Technol J, 13, No 2, pp 94—105 (April 1995).

2 Meyer B: 'Object oriented software construction', Prentice Hall (1988).

3 Cusack E L and Cordingley E S (Eds): 'Object oriented technology', BT Technol J, 11, No 3 (July 1993).

4 Lewis T J: 'Where is computing headed?' IEEE Computing Journal (August 1994).

5 Smith D G: 'Personal intelligent communications', BT Technol J, 13, No 2, pp 106—112 (April 1995).

6 Rees R T O: 'The ANSA computational model', AR.001, APM, Poseidon House, Castle Park, Cambridge CB3 0RD, England (1993).

7 Birman K P: 'The process group approach to reliable distributed computing', Communications of the ACM (December 1993).

8 Microsoft Corporation, One Microsoft Way, Redmond, WA 98052-6399.

9 Elbert B and Martyna B: 'Client server computing', Artech House (1994).

10 OSF: Open Software Foundation, 11 Cambridge Centre, Cambridge, MA 02142.

11 OMG: Object Management Group, Inc., Framlingham Corp Centre, 492 Old Connecticut Path, Framlingham, MA 01701.

12 Open Look, XView Reference Manual, O'Reilly and Associates, Inc.

13 ONTOS DBMS, Ontologic (1992).

14 ObjectStore, Object Design Inc., 25 Mall Road, Burlington, MA 01803-4194.

Paul Martin graduated from Bath in 1982 with a BSc in mechanical engineering. After this he entered the CAD/CAM industry with posts at Rolls Royce, PA management consultants and CADCentre.

He joined BT in 1991 to research the use of object oriented modelling techniques in communications.

Currently his main research interest is in the application of distributed objects technology to telecommunications systems.